

```
ROLLBACK
forumdb=> INSERT INTO tags( tag ) VALUES ( 'IntelliJIDEa IDE' );
INSERT 0 1
forumdb=> COMMIT;
COMMIT

forumdb=> SELECT tag FROM tags WHERE tag like '%IDE';
      tag
-----
Eclipse IDE
IntelliJIDEa IDE
(2 rows)
```

In the preceding transaction, the first statement does not belong to any savepoint and, therefore, follows the life of the explicit transaction itself. After the other_tags savepoint is created, all the following statements follow the life cycle of the savepoint itself; therefore, once ROLLBACK TO SAVEPOINT is issued, the statements within the savepoint are discarded. After that, other statements belong to the explicit transaction and, therefore, follow the life cycle of the transaction itself. Ultimately, the result is that everything that has been executed outside the savepoint is stored in the table.

Once you have defined a savepoint, you can also change your mind and release it, so that statements within the savepoint follow the same life cycle of the main transaction. Here's an example:

```
forumdb=> BEGIN;
BEGIN
forumdb=> SAVEPOINT editors;
SAVEPOINT
forumdb=> INSERT INTO tags( tag ) VALUES ( 'Emacs Editor' );
INSERT 0 1
forumdb=> INSERT INTO tags( tag ) VALUES ( 'Vi Editor' );
INSERT 0 1
forumdb=> RELEASE SAVEPOINT editors;
RELEASE
forumdb=> INSERT INTO tags( tag ) VALUES ( 'Atom Editor' );
INSERT 0 1
forumdb=> COMMIT;
```

```
COMMIT

forumdb=> SELECT tag FROM tags WHERE tag LIKE '%Editor';
      tag
-----
Emacs Editor
Vi Editor
Atom Editor
(3 rows)
```

When `RELEASE SAVEPOINT` is issued, it is like the savepoint has disappeared, and therefore, the two `INSERT` statements follow the main transaction life cycle. In other words, it is like the savepoint has never been defined.

In a transaction, you can have multiple savepoints, but once you roll back one, you roll back all the savepoints that follow it:

```
forumdb=> BEGIN;
BEGIN
forumdb=> SAVEPOINT perl;
SAVEPOINT
forumdb=> INSERT INTO tags( tag ) VALUES ( 'Rakudo Compiler' );
INSERT 0 1
forumdb=> SAVEPOINT gcc;
SAVEPOINT
forumdb=> INSERT INTO tags( tag ) VALUES ( 'Gnu C Compiler' );
INSERT 0 1
forumdb=> ROLLBACK TO SAVEPOINT perl;
ROLLBACK
forumdb=> COMMIT;
COMMIT

forumdb=> SELECT tag FROM tags WHERE tag LIKE '%Compiler';
      tag
-----
(0 rows)
```

As you can see, even if a transaction has issued a `COMMIT`, everything that has been done after the `perl` savepoint, to which the transaction has rolled back, has been rolled back too.

In other words, rolling back to a savepoint means you roll back everything done after the declaration of such a savepoint.

Transactions can lead to a situation where the cluster is unable to proceed. These situations are named *deadlocks* and are described in the next section.

Deadlocks

A deadlock is an event that happens when different transactions depend on each other in a circular way. Deadlocks are, to some extent, normal events in a concurrent database environment and nothing an administrator should worry about, unless they become extremely frequent, meaning there is some dependency error in the applications and the transactions.

When a deadlock happens, there is no choice but to terminate the locked transactions. PostgreSQL has a very powerful deadlock detection engine that does exactly this job: it finds stalled transactions and, in the case of a deadlock, terminates them (producing a ROLLBACK).

In order to produce a deadlock, imagine two concurrent transactions applying changes to the very same tuples in a conflicting way. For example, the first transaction could do something like the following:

```
-- session 1
forumdb=> BEGIN;
BEGIN
forumdb=> SELECT txid_current();
    txid_current
-----
          4875
(1 row)

forumdb=> UPDATE tags SET tag = 'Perl 5'
          WHERE tag = 'perl';
UPDATE 1
```

And in the meantime, the other transaction performs the following:

```
-- session 2
forumdb=> BEGIN;
BEGIN
forumdb=> SELECT txid_current();
```

```

txid_current
-----
          4876
(1 row)

forumdb=> UPDATE tags SET tag = 'Java and Groovy'
          WHERE tag = 'java';
UPDATE 1

```

So far, both transactions have updated a single tuple without conflicting with each other. Now, imagine that the first transaction tries to modify the tuple that the other transaction has already changed; as we have already seen in previous examples, the transaction will remain locked, waiting to acquire the lock on the tuple:

```

-- session 1
forumdb=> UPDATE tags SET tag = 'The Java Language'
          WHERE tag = 'java';
-- locked

```

If the second transaction tries, on the other hand, to modify a tuple already touched by the first transaction, it will be locked, waiting for the lock acquisition:

```

-- session 2
forumdb=> UPDATE tags SET tag = 'Perl and Raku'
          WHERE tag = 'perl';
ERROR:  deadlock detected
DETAIL:  Process 78918 waits for ShareLock on transaction 4875; blocked by
process 80105.
Process 80105 waits for ShareLock on transaction 4876; blocked by process
78918.
HINT:  See server log for query details.
CONTEXT:  while updating tuple (0,1) in relation "tags"

```

This time, however, PostgreSQL realizes the two transactions cannot solve the problem because they are waiting on a circular dependency, and therefore, it decides to kill the second transaction in order to give the first one a chance to complete. As you can see from the error message, PostgreSQL knows that transaction 4875 is waiting for a lock held by transaction 4876 and vice versa, so to proceed, there is no solution but to kill one of the two.

Being natural events in a concurrent transactional system, deadlocks are something you have to deal with, and your applications must be prepared to replay a transaction in case they are forced to ROLLBACK by deadlock detection.

Deadlock detection is a complex and resource-expensive process; therefore, PostgreSQL does it on a scheduled basis. In particular, the `deadlock_timeout` configuration parameter expresses how often PostgreSQL should search for dependency among stalled transactions. By default, this value is set at 1 second and is expressed in milliseconds:

```
forumdb=> SELECT name, setting, unit
           FROM pg_settings
           WHERE name like '%deadlock%';
   name      | setting | unit
-----+-----+-----
deadlock_timeout | 1000    | ms
(1 row)

forumdb=> SHOW deadlock_timeout;
deadlock_timeout
-----
1s
(1 row)
```

Decreasing this value is often a bad idea: your applications and transactions will fail sooner (if in a deadlock condition), but your cluster will be forced to consume extra resources in dependency analysis.

In the following section, you will discover how PostgreSQL ensures that data is made persistent on storage, even in the case of a crashing cluster.

How PostgreSQL handles persistency and consistency: WALs

In the previous sections, you have seen how to interact with transactions and, most notably, how PostgreSQL executes every statement within a transaction, either explicitly or implicitly.

PostgreSQL does, internally, very complex work to ensure that consolidated data in storage reflects the status of the committed transactions. In other words, data can be considered consolidated only if the transaction that produced (or modified) it has been successfully committed. But this also means that, once a transaction has been successfully committed, its data is “safe” on storage, no matter what happens in the future: if a transaction is reported to be successful, its data must be made persistent, even if the database or the whole system crashes.

PostgreSQL manages transactions and data consolidations by means of WALs. This section introduces you to the concept of WALs and their use within PostgreSQL.

WALs

Before we dig into the details, it is required to briefly explain how PostgreSQL internally handles data. Tuples are stored in mass storage – usually, a disk – under the `$PGDATA/base` directory, in files named only by numbers. When a transaction requests access to a particular set of tuples, PostgreSQL loads the requested data from the `$PGDATA/base` directory and places it in one or more shared buffers. The shared buffers are an in-memory copy of the on-disk data, and all the transactions access the shared data, because they provide much more performance and do not require every single transaction to seek the data out of the storage.

The next figure shows a few data pages loading into the shared buffers' memory location:

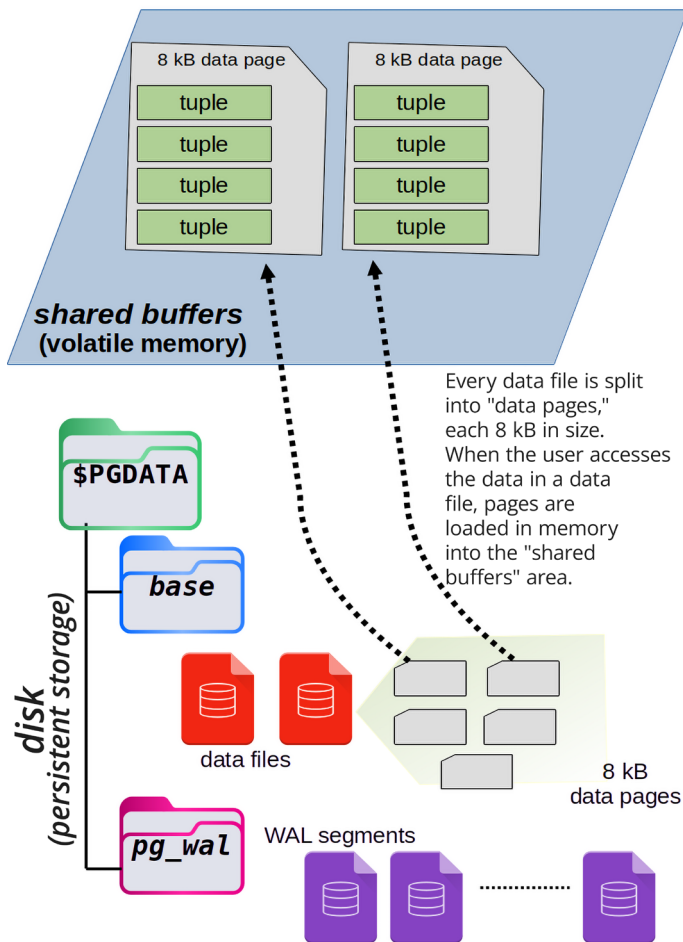


Figure 11.2: Loading data pages into the shared buffers' memory location

When a transaction modifies some data, it does so by modifying the in-copy memory, which means it modifies the “shared buffers” area.

At this point, the in-memory copy of the data does not correspond to the stored version, and it is here that PostgreSQL has to guarantee consistency and persistency without losing performance.

What happens is that the data is kept in memory but is marked as dirty, meaning that it is a copy not yet synchronized with the on-disk original source. Once the changes to a dirty buffer have been committed, PostgreSQL consolidates the changes in the WALs and keeps the dirty buffer in memory, to be served as the most recent available copy for other transactions and connections.

Sooner or later, PostgreSQL will push the dirty buffer to the storage, replacing the original copy with the modified version, but a transaction usually does not know and care about when this is going to happen.

The following diagram explains the preceding workflow: the red buffer has been modified by a transaction and, therefore, does not match what is on disk anymore. However, when the transaction issues a COMMIT, the changes are forced and flushed to the WALs:

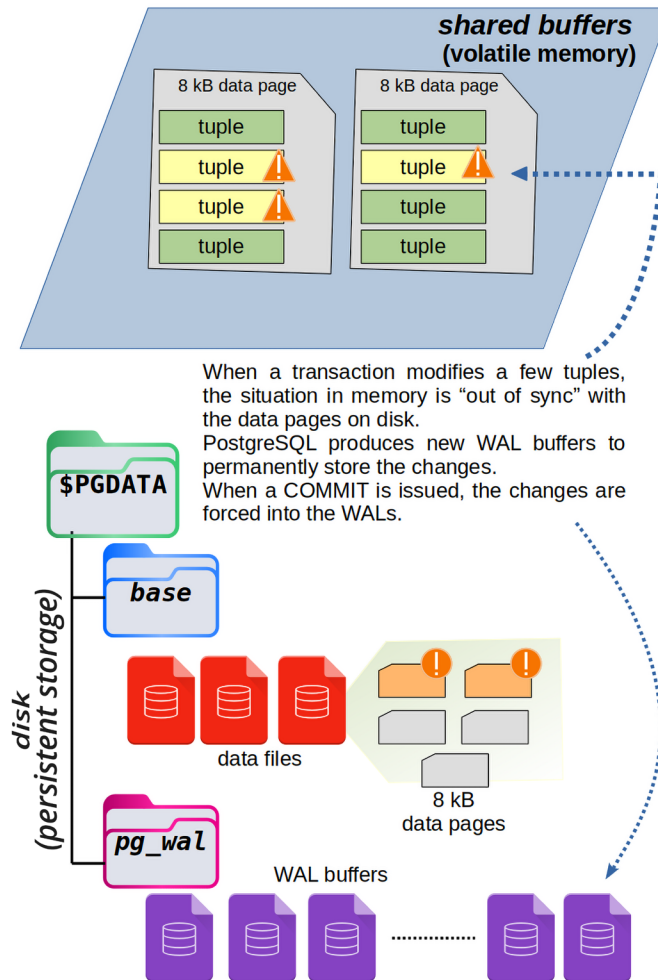


Figure 11.3: After a COMMIT, changes are forced into the WALs

Why is the WAL space considered to be more efficient than overwriting the original data block in the `$PGDATA/base` directory? The trick is that in order to find the exact position on the disk storage where the block has to be overwritten, PostgreSQL should have to perform what is called a random-seek, which is a costly I/O operation. On the other hand, WALs are sequentially written as a journal, and therefore, there is no need to perform a random-seek. Writing the WALs prevents the I/O performance degradation and allows PostgreSQL to overwrite the data block in the future, when, for instance, the cluster is not overloaded and has I/O bandwidth available.

Every time a transaction performs a `COMMIT`, its actions and modified data are permanently stored in a piece of the WAL, in particular a specific part of the current WAL segment (more on this later). Therefore, PostgreSQL can reproduce the transaction and its effects in order to perform the very same data changes.

This, however, does not suffice in making PostgreSQL reliable: PostgreSQL makes a big effort to ensure data actually hits the disk storage. In particular, during the writing of the WALs, PostgreSQL isolates itself from the outside world, disabling operating system signals, so that it cannot be interrupted. Moreover, PostgreSQL issues `fsync(2)`, a particular operating system call that forces the filesystem cache to flush data on disk.

PostgreSQL does all of this in order to ensure that the data physically hits the disk layer, but it must be clear that if the filesystem, or the disk controller (that is, the hardware), lies, the data could not be physically on the disk. This is important, but PostgreSQL cannot do anything about that and has to trust what the operating system (and, thus, the hardware) reports back as feedback.

In any case, `COMMIT` will return success to the invoking transaction if, and only if, PostgreSQL has been able to write the changes on the disk. Therefore, at the transaction level, if a `COMMIT` succeeds (that is, there is no error), the data has been written in the WALs and, therefore, can be assumed to be *safe* on the storage layer.

WALs are split into chunks called *segments*. A segment is a file made of exactly 16 MB of changes in data. While it is possible to modify the size of segments during `initdb`, we strongly discourage this and will assume every segment is 16 MB in size.

This means that PostgreSQL writes, sequentially, a single file at a time (that is, a WAL segment), and when this has reached the size of 16 MB, it is closed and a new 16 MB file is created. The WAL segments are stored in the `pg_wal` directory under `$PGDATA`. Every segment has a name made up of hexadecimal digits, 24 characters long.

The first eight characters indicate the so-called *timeline* of the cluster (something related to physical backups and replication), the next eight digits indicate an increasing sequence number named the **Log Sequence Number (LSN)**, and the last eight digits provide the offset within the LSN. Here's an example:

```
$ ls -l $PGDATA/pg_wal
0000000700000247000000A8
0000000700000247000000A9
0000000700000247000000AA
0000000700000247000000AB
0000000700000247000000AC
0000000700000247000000AD
...
```

In the previous content of the `pg_wal`, you can see that every WAL segment has the same timeline, number 7, and the LSN is 247. Every file, then, has a different offset, with the first one being A8, the second A9, and so on. As you can imagine, WAL segment names are not made for humans, but PostgreSQL knows exactly how and in which file it has to search for information.

Sooner or later, depending on the memory resources and usage of the cluster, the data in memory will be written back to its original disk positions, meaning that the WALs serve only as temporary safe storage on disk. The reason for that is not only tied to a performance boost, as already explained, but also to allow data restoration in the event of a crash.

WALs as a rescue method in the event of a crash

When you cleanly stop a running cluster, for example, by means of `pg_ctl`, PostgreSQL ensures that all dirty data in memory is flushed to the storage in the correct order, and then halts itself.

But what happens if the cluster is uncleanly stopped, for example, by means of a power failure?

This event is named a **crash**, and once PostgreSQL starts over, it performs a so-called **crash recovery**. In particular, PostgreSQL understands it has stopped in an unclean way, and therefore, the data on the storage might not be the last version that existed in memory when the cluster terminated its activity. But PostgreSQL knows that all committed data is at least present in the WALs and, therefore, starts reading the WALs in what is called **WAL replay**, adjusting the data in the storage according to what is in the WALs. Until the crash recovery has completed, the cluster is not usable and does not accept connections; once the crash recovery has finished, the cluster knows that the data on the storage has been made coherent, and therefore, normal operativity can start again.